Comprehensive Technical & Conceptual Architecture Report: Memossist

Contents

1. Executive Summary & Conceptual Vision 1
2. Problem Statement 1
3. Core Solutions & System Architecture 2
4. Mathematical Formulations & Core Algorithms 4
5. End-to-End Data Flow & Execution Lifecycle 8
6. Summary of Architectural Advantages 9

1. Executive Summary & Conceptual Vision

Memossist is an autonomous, on-device cognitive memory architecture and personal AI assistant for mobile operating systems (Android). It transforms ephemeral, stateless Large Language Model (LLM) chats into a **stateful, evolving, biologically-inspired long-term digital memory vault**.

Unlike traditional cloud-based digital assistants that treat every session as an isolated query or stream raw personal telemetry to remote datacenters, Memossist operates as a **closed-loop, self-sovereign cognitive system**. It combines:

1. **On-device neural inference** (via native ARM64-optimized quantized LLMs).
2. **Deterministic cognitive memory modeling** (Ebbinghaus-inspired exponential forgetting, synaptic reinforcement, and automated pruning).
3. **Graph-theoretic associative knowledge structures** (Experience Directed Acyclic Graph / DAG with Hebbian-like co-activation updates).
4. **Deterministic sub-millisecond local NLP classifiers** (TF-IDF + Multinomial Logistic Regression).
5. **Multi-stage temporal proactive schedulers** (exact hardware alarms with automated multi-point lifecycle reminders).

---

2. Problem Statement

2.1 The Cloud AI Privacy Paradox

Modern LLM assistants require rich personal context (daily schedules, health notes, relationships, private thoughts) to provide personalized utility. However, transmitting this raw biographical data to remote cloud servers exposes users to surveillance, data mining, breach vulnerabilities, and vendor lock-in.

2.2 Context Window Limitations & Context Fragmentation

Standard LLMs suffer from statelessness and finite context window sizes. While Retrieval-Augmented Generation (RAG) exists, standard vector embeddings running on mobile devices incur high memory overhead, generate hallucinations over stale data, and lack the ability to naturally forget obsolete information.

2.3 Static vs. Dynamic Memory Modeling

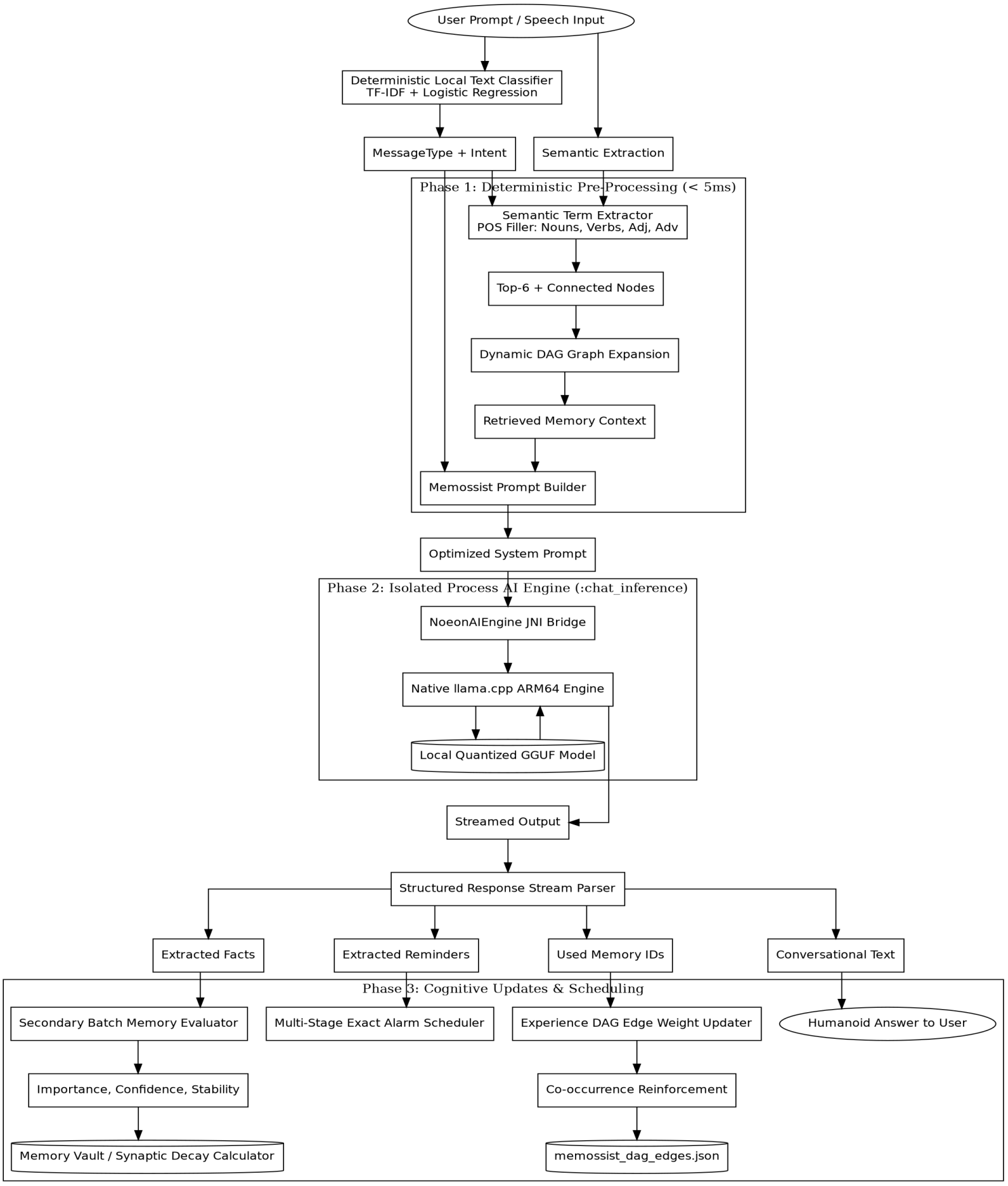
Traditional note-taking and reminder applications are static repositories: an entry written three years ago retains the same relevance weight as an entry made yesterday. In human cognition, memories undergo **continuous synaptic plasticity**: frequently recalled memories strengthen and form associative webs, while irrelevant or unreinforced memories decay and are forgotten.

2.4 Mobile Hardware & Latency Constraints

Executing multi-billion parameter neural networks on edge hardware (3GB–8GB RAM ARM64 devices) introduces strict thermal, memory, and OS lifecycle boundaries. Android's aggressive Low Memory Killer (LMK) and OEM background process management terminate memory-intensive tasks unless architectural isolation and strict computation budgeting are enforced.

---

3. Core Solutions & System Architecture


User Prompt / Speech Input
Deterministic Local Text Classifier
TF-IDF + Logistic Regression
MessageType + Intent
Semantic Extraction
Phase 1: Deterministic Pre-Processing (< 5ms)
Semantic Term Extractor
POS Filler: Nouns, Verbs, Adj, Adv
Top-6 + Connected Nodes
Dynamic DAG Graph Expansion
Retrieved Memory Context
Memossist Prompt Builder
Optimized System Prompt
Phase 2: Isolated Process AI Engine (:chat_inference)
NoeonAIEngine JNI Bridge
Native llama.cpp ARM64 Engine
Local Quantized GGUF Model
Streamed Output
Structured Response Stream Parser
Extracted Facts
Extracted Reminders
Used Memory IDs
Conversational Text
Phase 3: Cognitive Updates & Scheduling
Secondary Batch Memory Evaluator
Multi-Stage Exact Alarm Scheduler
Experience DAG Edge Weight Updater
Humanoid Answer to User
Importance, Confidence, Stability
Co-occurrence Reinforcement
Memory Vault / Synaptic Decay Calculator
memossist_dag_edges.json

---

4. Mathematical Formulations & Core Algorithms

4.1 Biological Memory Decay & Forgetting Engine

Memossist models human episodic memory using mathematical formulations inspired by the **Ebbinghaus Forgetting Curve** combined with **synaptic consolidation theory**.

Every memory item M_k stored in the vault contains dynamic cognitive parameters:

- $I \in [0.0, 1.0]$: **Importance** (intrinsic value of the memory).
- $C \in [0.0, 1.0]$: **Confidence** (certainty of the stated knowledge).
- $T \in [0.0, 1.0]$: **Stability** (temporal permanence/longevity of the fact).
- $S_{\text{base}} \in [0.0, 1.0]$: **Base Synaptic Strength**.
- $A \in \mathbb{N}_0$: **Access Count** (number of times retrieved).
- $R_c \in \mathbb{N}_0$: **Reinforcement Count** (number of times reinforced by user or LLM).
- t_{last}: Timestamp of the last reinforcement event.

A. Initial Memory Strength (S_0) When a new memory is extracted, its initial base strength S_0 is computed via a weighted linear combination:

$$S_0 = \text{clamp}\Big(w_I \cdot I + w_C \cdot C + w_T \cdot T,\ 0.0,\ 1.0\Big)$$

$$\text{where } w_I = 0.50, \quad w_C = 0.20, \quad w_T = 0.30$$

B. Dynamic Half-Life (H) The retention half-life H (in days) determines the resistance of a memory to decay:

$$H = H_{\text{base}} + k_I \cdot I + k_T \cdot T$$

$$\text{where } H_{\text{base}} = 7.0 \text{ days}, \quad k_I = 120.0, \quad k_T = 120.0$$

A high-importance, high-stability memory can achieve a half-life of up to 247 days, whereas an ephemeral detail decays rapidly ($H \approx 7$ days).

C. Exponential Time Decay Factor (F_{time}) Given the elapsed days $D = \frac{t_{\text{now}} - t_{\text{last}}}{86{,}400{,}000 \text{ ms}}$:

$$F_{\text{time}} = 2^{-\frac{D}{H}}$$

D. Non-Linear Diminishing Access & Reinforcement Boosts Repeated accesses and reinforcements strengthen synaptic links with logarithmic diminishing returns to prevent runaway saturation:

$$B_{\text{access}} = 1.0 + \alpha \cdot \ln(1.0 + A), \quad \alpha = 0.15$$

$$B_{\text{reinforce}} = 1.0 + \beta \cdot \ln(1.0 + R_c), \quad \beta = 0.25$$

E. Instantaneous Strength Calculation (S_{current})

$$S_{\text{current}} = \text{clamp}\Big(S_{\text{base}} \cdot F_{\text{time}} \cdot B_{\text{access}} \cdot B_{\text{reinforce}},\ 0.0,\ 1.0\Big)$$

F. Synaptic Plasticity & Reinforcement Rule When an existing memory is accessed during reasoning or confirmed by the user, its base strength updates:

$$\Delta R = 0.10 + 0.10 \cdot I$$
$$S_{\text{base}} \leftarrow \min(1.0,\ S_{\text{base}} + \Delta R)$$
$$R_c \leftarrow R_c + 1, \quad t_{\text{last}} \leftarrow t_{\text{now}}$$

G. Automated Forgetting & Pruning Condition A background periodic evaluator (`MemoryDecayWorker`) calculates S_{current} across all vault memories. Memories falling below the critical threshold are autonomously culled:

$$\text{Decision}(M_k) = \begin{cases} \text{FORGET (Prune)}, & \text{if } S_{\text{current}} < 0.15 \\ \text{KEEP}, & \text{if } S_{\text{current}} \geq 0.15 \end{cases}$$

4.2 Associative Experience DAG (Directed Acyclic Graph)

Memories are not stored in flat isolation; they form a **topological semantic graph** $G = (V, E)$, where each vertex $v_i \in V$ represents a discrete memory item, and each edge $e_{ij} \in E$ holds an associative strength $S_{ij} \in [0.0, \infty)$.

```
[Memory 1: "Morning Workout"]
            \
             \ Edge Strength: S_12 = S_old + (C * t) / N
              \
[Memory 2: "Protein Diet Plan"] ----- [Memory 3: "Doctor Advice"]
```

A. Linguistic Content Filtering To ensure topological links reflect semantic relationships rather than syntactic noise, all text is filtered through a Part-of-Speech (POS) content extractor (`LinguisticAnalyzer`).

- **Excluded Non-Content Words**: Pronouns, Prepositions, Conjunctions, Determiners, Auxiliary/Modal Verbs, Conversational Fillers.
- **Retained Semantic Vocabulary**: Strictly Nouns, Action Verbs, Descriptive Adjectives, and Contextual Adverbs expanded with local morphological synonym dictionaries.

For any text snippet X, let $\mathrm{T}(X)$ denote its filtered semantic word-and-synonym set:

$$\mathrm{T}(X) = \bigcup_{w \in X} \Big(\{w\} \cup \text{Synonyms}(w) \Big) \cap \mathrm{V}_{\text{content}}$$

B. Hebbian-like Co-Occurrence Edge Learning Formula When the user submits a prompt Q, generating candidate set $\mathrm{K} = \{M_1, M_2, \dots, M_k\}$ where a subset $\mathrm{U} \subseteq \mathrm{K}$ is actively utilized in synthesizing the response:

1. **Question Semantic Set**: $Q_{\text{terms}} = \mathrm{T}(Q)$
2. **Global Candidate Union Scope (N)**:

$$N = \left| \bigcup_{M_k \in \mathrm{K}} \mathrm{T}(M_k) \right|$$

3. **Pairwise Shared Context Intersection (C_{ij})**:

$$C_{ij} = \left| Q_{\text{terms}} \cap \mathrm{T}(M_i) \cap \mathrm{T}(M_j) \right|$$

4. **Edge Incremental Weight (ΔS_{ij})**:

$$\Delta S_{ij} = \frac{C_{ij} \cdot t_{ij}}{N}$$

$$\text{where } t_{ij} = \begin{cases} 1, & \text{if } M_i \in \mathrm{U} \wedge M_j \in \mathrm{U} \\ 0, & \text{otherwise} \end{cases}$$

5. **Edge Update**:

$$S_{ij}^{(\text{new})} = S_{ij}^{(\text{old})} + \Delta S_{ij}$$

C. Dynamic Graph Expansion Retrieval When retrieving context for a new query:

1. Rank all memories by semantic overlap $|Q_{\text{terms}} \cap \mathrm{T}(M_k)|$, selecting the Top-5 primary candidates: $(M_{(1)}, M_{(2)}, \ldots, M_{(5)})$.
2. For each candidate at rank $i \in \{1, 2, 3, 4, 5\}$, expand the retrieval neighborhood by traversing connected edges:

$$\text{Fetch Count}(i) = 5 - i$$

   Top $(5 - i)$ connected nodes M_c with $S_{i,c} > 0.0$ (sorted descending by edge strength) are pulled into working memory.
3. Execute rank-preserving union deduplication:

$$\mathrm{C}_{\text{final}} = \text{UniqueByOrder}\Big([M_{(1)}, \text{Neighbors}(M_{(1)}), M_{(2)}, \text{Neighbors}(M_{(2)}), \ldots]\Big)$$

4.3 Deterministic Sub-Millisecond Text Classification

To eliminate the latency and token overhead of asking an LLM to classify its own inputs, Memossist incorporates an on-device TF-IDF vectorizer paired with a trained Multinomial Logistic Regression model.

```
Raw Text ---> Tokenizer (Unigram + Bigram) ---> Sublinear TF-IDF ---> L2 Normalizer ---> Softmax Boundary ---> MessageType
```

A. Feature Extraction & Sublinear Scaling Given input text x, tokens are extracted as unigrams and adjacent bigrams matching word boundaries:

$$\text{Tokens}(x) = \{w_k\} \cup \{w_k w_{k+1}\}$$

For each token t in vocabulary V:

$$\mathrm{TF}(t, x) = \text{count}(t, x)$$

$$\mathrm{TF}_{\text{sublinear}}(t, x) = 1.0 + \ln(\mathrm{TF}(t, x)) \quad (\forall \mathrm{TF} > 0)$$

$$\text{Weight}(t, x) = \mathrm{TF}_{\text{sublinear}}(t, x) \cdot \text{IDF}(t)$$

B. L_2 Euclidean Vector Normalization

$$\mathbf{v}(x) = \frac{\mathbf{w}(x)}{\|\mathbf{w}(x)\|_2} = \frac{\mathbf{w}(x)}{\sqrt{\sum_{t \in \mathrm{V}} \mathbf{w}(x)_t^2}}$$

C. Decision Boundaries & Probability Calibration

- **Binary Classifier (Reminder Detection)**:

$$P(\text{IsReminder} \mid x) = \sigma(\mathbf{w}^T \mathbf{v}(x) + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{v}(x) + b)}}$$

- **Multiclass Classifier (Intent Routing: Asking vs. Telling vs. Mixed)**:

$$P(y = k \mid x) = \frac{e^{\mathbf{w}_k^T \mathbf{v}(x) + b_k}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^T \mathbf{v}(x) + b_j}}$$

The classification result instantly determines which of the **7 specialized, minimal system prompts** is built, eliminating unnecessary prompt instructions and reducing generation prefill latency by up to 60%.

4.4 Multi-Stage Proactive Temporal Scheduler

When reminders are extracted, Memossist constructs a scheduled lifecycle rather than a single alert. Given target timestamp T_{event}:

Trigger Phase	Timestamp Calculation	Delivery Mode	Objective
Phase 1: Advance Notice	$T - 24$ hours	System Notification	Early preparation awareness
Phase 2: Morning Briefing	Day(T) at 07:00 AM	Interactive Notification	Daily executive orientation
Phase 3: Immediate Prep	$T - 1$ hour	High-Priority Notification	Transition and departure buffer
Phase 4: Critical Execution	$T - 10$ minutes	Full-Screen System Alarm	Urgent intervention (`AlarmAlertActivity`)
Phase 5: Cognitive Follow-up	$T + 45$ minutes	Conversational Notification	Outcome confirmation & episodic logging

All triggers utilize Android's `AlarmManager.setExactAndAllowWhileIdle()` to ensure execution across Doze mode.

4.5 2D Topological Graph Visualization Engine

To visualize high-dimensional memory relationships on a 2D canvas (`DagGraph2DView`), Memossist combines **Fermat's Spiral Phyllotaxis** with **Euler-integrated inertia panning**:

A. Node Coordinate Initialization (Golden Angle Spiral) To prevent visual node overlapping without heavy iterative physics settling on mobile CPUs, nodes are positioned via:

$$\theta_i = i \cdot \Phi_{\text{golden}} = i \cdot 2.39996323 \text{ rad} \quad (137.5077^\circ)$$

$$r_i = k_{\text{spacing}} \cdot \sqrt{i} + \delta_{\text{jitter}}$$

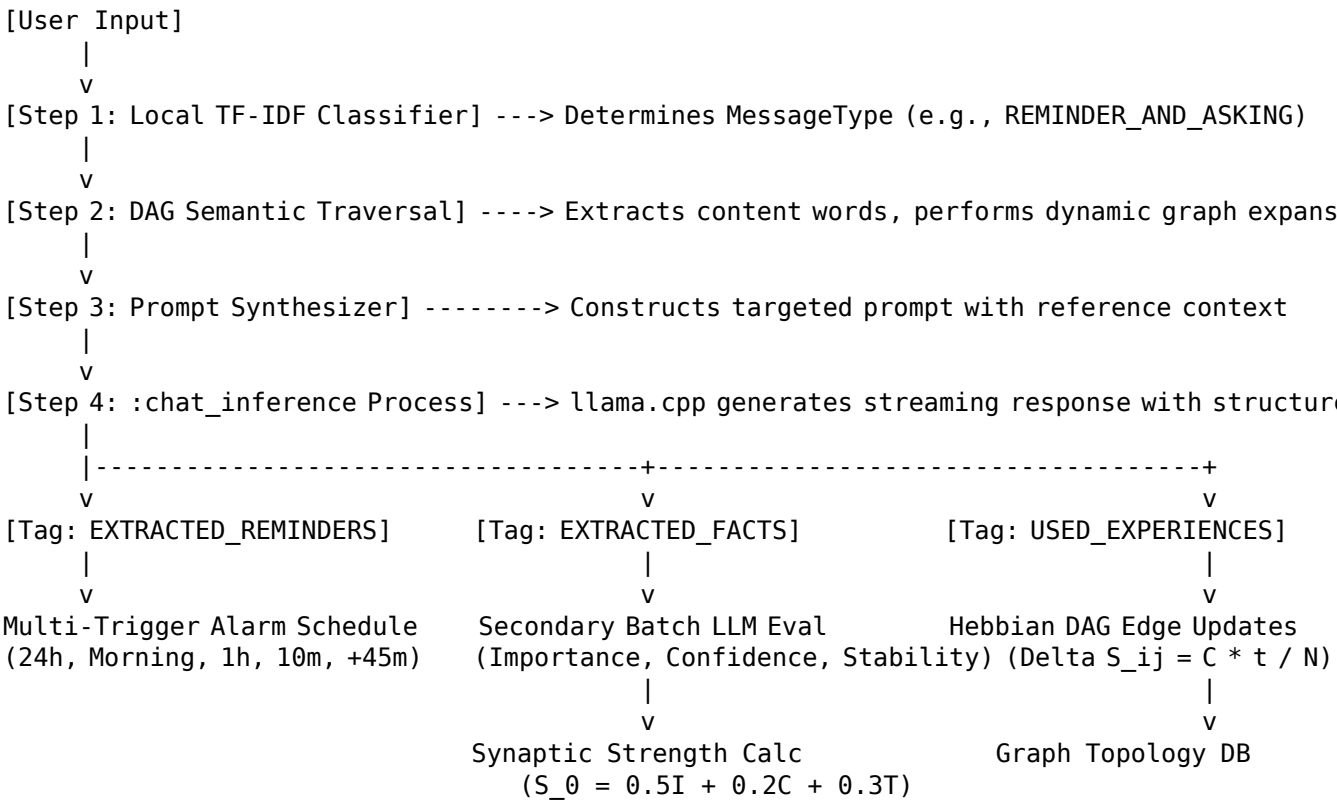
$$x_i = r_i \cdot \cos(\theta_i), \quad y_i = r_i \cdot \sin(\theta_i)$$

B. Dynamic Edge Rendering Edge line thickness and opacity are rendered proportional to mathematical strength:

$$\text{StrokeWidth}(e_{ij}) = \text{clamp}(2.0 + 4.0 \cdot S_{ij},\ 2.0,\ 10.0)\ \text{px}$$

---

5. End-to-End Data Flow & Execution Lifecycle



1. **Message Ingestion & Local Pre-Classification**: The raw user string is passed to `LocalTextClassifier`. In under 3ms, the system computes the exact classification (`REMINDER_ONLY`, `ASKING`, `TELLING`, `MIXED`, or combinations).
2. **Associative Graph Retrieval**: Content words are extracted. The graph retrieval algorithm pulls the top candidate memories and traverses the highest-weight connected edges.
3. **Context-Optimized Prompt Synthesis**: `MemossistPromptBuilder` injects current temporal anchors (`CURRENT REFERENCE DATE & TIME`) and candidate memories into a strict schema.
4. **Isolated Neural Execution**: The prompt is dispatched across process boundaries to `:chat_inference`. Native `llama.cpp` generates tokens using 4-bit / 8-bit quantized models without blocking the main UI thread.
5. **Tag-Delimited Stream Demuxing**: As output streams, headers are parsed:
   - `[EXTRACTED_REMINDERS]`: Parsed into structured dates, times, and categories; registered in Android's `AlarmManager`.
   - `[EXTRACTED_FACTS]`: Enqueued in `MemoryParameterQueueManager`. A secondary background evaluation job calculates (I, C, T) parameters and registers new base strengths.
   - `[USED_EXPERIENCES]`: Active memory IDs trigger edge updates ($\Delta S_{ij} = \frac{C \cdot t}{N}$) in the DAG database and increment reinforcement counters.

- `[HUMANOID_ANSWER]`: Filtered clean conversational text is rendered directly to the user interface.

---

6. Summary of Architectural Advantages

Dimension	Standard Cloud Assistants	Conventional Mobile Note/RAG Apps	Memossist Architecture
Data Privacy	Cloud servers; telemetry harvesting	Local storage, but no intelligence	**100% On-Device; zero network telemetry**
Inference Engine	Remote API servers	Heavy mobile cloud APIs	**Native C++ `llama.cpp` JNI on ARM64**
Memory Mechanics	Stateless or infinite flat history	Static database records	**Ebbinghaus exponential decay + forgetting**
Knowledge Topology	Vector database similarity only	Manual folders/tags	**Associative 2D Experience DAG with Hebbian learning**
Intent Processing	Full prompt roundtrip to cloud LLM	Hardcoded keyword heuristics	**Sub-millisecond TF-IDF + Logistic Regression**
Proactive Alarms	Single generic push notification	Single static calendar alert	**5-stage automated temporal lifecycle scheduler**